Titanic: Machine Learning from Disaster

System Analysis Project

Julián Carvajal Garnica
20242020024

Andrés Mauricio Cepeda Villanueva
20242020010

Jhonatan David Moreno Barragán
20201020094

Andrés Camilo Ramos Rojas
20242020005

School of Engineering, Universidad Distrital Francisco José de Caldas
System Analysis Course
Teacher: Carlos Andrés Sierra
Bogotá D.C.
2025

Part 2 of the System Analysis Project

1. Review of Workshop #1 Findings

This section summarizes the main analytical outcomes from Workshop #1 and establishes the foundation for the system design developed in Workshop #2. All insights are derived exclusively from the Kaggle dataset *"Titanic: Machine Learning from Disaster"*, focusing strictly on the structure, dependencies, and behavior observable within the data.

1.1 Summary of System Characteristics Identified in Workshop #1

The Titanic dataset was defined as a closed input–output system where each instance (passenger) contains structured attributes determining the binary outcome `Survived`. Workshop #1 identified the following essential characteristics:

- **Input Variables (System Elements):** Passenger information such as `Sex`, `Age`, `Pclass`, `SibSp`, `Parch`, `Fare`, `Cabin`, `Embarked`, and name-derived titles.
- **Output Variable:** `Survived` (0 or 1), representing the system's final state.
- **Internal Relations:** The dataset exhibits structural relationships, such as:
  - `Pclass` correlates with `Fare`.
  - `Sex` correlates with `Survived`.
  - `Age` patterns vary across passenger groups.
  - `SibSp + Parch` form implicit family structures.
  - `Embarked` distribution relates to `Pclass` and `Fare`.
- **Constraints:**
  - Missing values in `Age`, `Cabin`, and occasionally `Embarked`.
  - High variability in string-based variables such as `Name` and `Ticket`.
  - Class imbalance between survivors and non-survivors.

- **Sensitivity:** Small modifications to key features (e.g., `Age`, `Sex`, or `Pclass`) have a significant impact on the target output due to their strong correlations.
- **Observed Complexity:** Feature interactions are nonlinear, and certain combinations of variables produce inconsistent or unpredictable patterns (e.g., passengers with nearly identical profiles having opposite survival outcomes).
- **Dataset-Level Randomness:** Variability in `Cabin` availability, unusual ticket formats, and outliers in `Fare` introduce irregularities into the dataset.

1.2 Implications for System Design

These analytical findings directly influence the architectural decisions required for Workshop #2:

- **The system must manage missing data** through preprocessing steps such as imputation or removal.
- **Variable encoding is essential**, especially for `Sex`, `Embarked`, and name-derived titles.
- **Modeling components must address feature nonlinearity** (e.g., through tree-based models or engineered features).
- **The design must incorporate sensitivity handling:** highly impactful variables require monitoring during modeling and evaluation.
- **Chaos-handling mechanisms arise naturally from the dataset:** unpredictable or inconsistent instances require robust evaluation and possibly ensemble models.
- **The architecture must be modular:** separating ingestion, preprocessing, modeling, evaluation, and output generation aligns with the dataset's internal structure and ensures maintainability.

1.3 Transition Toward Workshop #2 Design

Workshop #1 provided a systemic understanding of the data and its internal dynamics. Workshop #2 builds on this foundation by transforming that analytic perspective into a structured, engineering-oriented system design. The insights above guide the requirements, architecture, and implementation strategy developed in the following sections.

2. System Requirements

This section translates the insights obtained in Workshop #1 into concrete and measurable design requirements. All requirements are derived strictly from the structure, limitations, and behavior of the Kaggle Titanic dataset, as analyzed previously (e.g., missing values, categorical dependencies, variable sensitivity, and internal relationships).

2.1 Translation of Analysis Findings into Design Requirements

Based on the systemic analysis, the system must satisfy requirements that directly address the dataset's characteristics:

- **R1 — Handling Missing Data (Reliability):** Since variables such as `Age` and `Cabin` contain missing values, the system must include robust imputation or exclusion mechanisms to prevent instability in downstream processing.
- **R2 — Consistent Categorical Processing (Performance & Accuracy):** The model must standardize categorical features (e.g., `Sex`, `Embarked`, `Pclass`) using deterministic encoding strategies to reduce variability and ensure reproducibility.
- **R3 — Management of Sensitive Attributes (Stability):** Variables identified as highly sensitive in Workshop #1 (e.g., `Sex`, `Pclass`, `Age`) must be treated with stable transformations that avoid amplification of small perturbations.
- **R4 — Feature Interaction Support (Model Robustness):** The system must support engineered features derived from dataset relationships, such as `FamilySize = SibSp + Parch + 1` or titles extracted from `Name`, since these reflect internal structure observed in the dataset.
- **R5 — Traceability and Repeatability (Maintainability):** All preprocessing steps must be versioned so that the same transformation pipeline can be recreated consistently.
- **R6 — Evaluation Consistency (Quality Assurance):** The model evaluation process must compute accuracy in alignment with Kaggle requirements, ensuring comparability with the competition's scoring mechanism.

These requirements ensure that the system design is directly aligned with the dataset's internal constraints and sensitivities.

2.2 User-Centric Needs

Although the system is primarily data-driven, several user-focused considerations remain relevant:

- **U1 — Interpretability:** Users (students or analysts) must be able to understand how inputs such as `Sex`, `Pclass`, and `FamilySize` affect the prediction. This may require simple model explanations or transparency of preprocessing steps.
- **U2 — Ease of Use:** The system should allow users to run the preprocessing and modeling pipeline with minimal configuration, following a clear sequence aligned with the architecture defined earlier.
- **U3 — Reproducibility for Learning:** Because this system is part of an academic project, users must be able to reproduce results consistently for validation and experimentation.
- **U4 — Data Safety (If Applicable):** Even though the dataset is public and anonymized, the system must avoid unintended modifications to source files (`train.csv`, `test.csv`), ensuring integrity during processing.

These user-centric requirements ensure that the system remains accessible, transparent, and reliable for those interacting with it.

3. High-Level Architecture

Based on the systemic analysis from Workshop #1, the proposed architecture evolves from a simple input-output model to a **robust, layered design** aligned with ISO 25010 software quality standards. This modular approach ensures that the system handles the dataset's intrinsic complexity, sensitivity, and missing information effectively.

The architecture is organized into three distinct layers, each responsible for a specific quality attribute of the system:

3.1 Reliability Layer (Data Integrity)

This layer acts as the system's defensive shield. Its primary function is to manage the "Constraints" and "Chaos" identified in the dataset (e.g., missing Age/Cabin, outliers in Fare).

- **Data Validation & Ingestion:** Receives raw data from `train.csv` and checks for structural consistency.

- **Imputation Module:** Handles missing values deterministically to prevent system instability.
- **Outlier Management:** Identifies and neutralizes inconsistent data points (such as extreme Fares in lower classes) before they propagate to the modeling stage.

3.2 Maintainability & Scalability Layer (Core Processing)

To address the requirement for modularity[cite: 43, 59], the transformation logic is decoupled into independent modules. This allows for specific adjustments without affecting the entire pipeline.

- **Transformation Pipeline:** A central orchestrator that manages data flow.
- **Feature Engineering Modules:**
  - *Text Processing:* Extracts titles from `Name` to capture social status.
  - *Numeric Processing:* Handles binning for `Age` and `Fare` to reduce sensitivity to small variations.
  - *Group Processing:* Combines `SibSp` and `Parch` to recreate family structures.
- **Unified Feature Vector:** Merges all processed signals into a standardized format ready for classification.

3.3 Usability Layer (Output & Interpretation)

Focusing on the user-centric requirements[cite: 66], this layer ensures that the system's output is not just a prediction, but an interpretable result.

- **Classification Logic:** Maps the unified features to the target variable using the trained model.
- **Output Generation:** Produces the final binary state (`Survived`: 0 or 1).
- **Confidence Metadata:** Provides auxiliary information (probability scores) to help users understand the certainty of the prediction, addressing the "Interpretability" requirement.

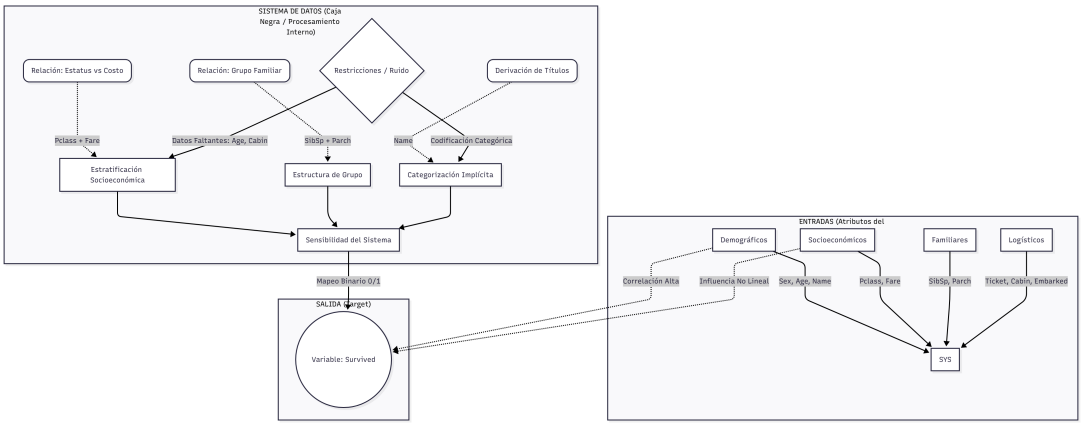

Figure 1: Robust System Architecture Diagram showing the Reliability, Maintainability, and Usability layers.

4. Addressing Sensitivity and Chaos

Based on the systemic behaviour identified in Workshop 1, the Titanic dataset exhibits sensitivity and chaotic characteristics that influence the stability of the predictive system. In this workshop, these findings are translated into concrete design strategies that reinforce robustness and reduce unpredictable system behaviour.

4.1 Sensitivity in the Dataset

The system is highly sensitive to variations in several input variables. Small changes in certain features produce disproportionately large effects on the output `Survived`. This sensitivity arises from the statistical structure of the dataset:

- **Sex**: This variable has a strong and nonlinear relationship with the target. Minor reassignments or misclassifications drastically change the prediction.
- **Age**: Missing values and continuous variation introduce instability. Small modifications after imputation significantly alter predicted probabilities.
- **Pclass**: Acts as a proxy for socioeconomic grouping; changes in class assignment modify multiple related behaviours (fare distribution, embarked patterns).
- **Cabin**: The irregularity and sparsity of this feature create fragile dynamics. When present, it carries strong information; when absent, imputations add uncertainty.

To mitigate sensitivity, the system design incorporates:

- Controlled imputation methods with versioning to avoid inconsistent behaviour.
- Normalization and encoding pipelines that stabilize variable transformations.

- Sensitivity-aware modeling (ensemble methods and cross-validation).

4.2 Chaotic Behaviour in the Dataset System

Although the dataset is static, the relationships among variables exhibit chaotic tendencies from a system dynamics perspective. These patterns do not arise from external historical factors, but from the internal structure of the data itself:

- **Outliers** in Fare and Age introduce unpredictable deviations.
- **Passengers with nearly identical profiles** sometimes have opposite survival outcomes, indicating internal variability that cannot be fully captured.
- **Irregular formats in Ticket and Cabin** create structural noise that propagates through preprocessing.
- **Nonlinear feature interactions** produce sudden changes in prediction when multiple variables shift slightly at the same time.

4.3 Design Strategies to Handle Chaos

To reduce the influence of chaotic elements, the architecture incorporates:

- **Feedback monitoring**: The Evaluation Layer continuously checks for unstable behaviours in the model's predictions.
- **Robust preprocessing**: Feature engineering and normalization limit the amplification of small anomalies.
- **Model ensembles**: Combining multiple models reduces the effect of unstable patterns present in any single model.
- **Performance drift detection**: If chaotic behaviour increases (e.g., sudden changes in evaluation metrics), the system flags the model and triggers a controlled retraining process.

4.4 Summary

The system design explicitly addresses the sensitivity and chaotic tendencies identified in Workshop 1. Through preprocessing controls, model stability techniques, and feedback mechanisms, the architecture maintains equilibrium and reduces unpredictable behaviour, ensuring a more reliable and resilient predictive system.

5. Technical Stack

The technical stack defines the set of tools and technologies that support each stage of the system architecture designed in Workshop 2. The selected stack reflects the requirements identified in the previous items: modularity, clarity, reproducibility, and the ability to manage sensitivity and variation in the Kaggle dataset.

5.1 Tools and Technologies

Data Layer:

- **Python** — Primary programming language used for the entire pipeline.
- **Pandas** — Data ingestion, loading of `train.csv` and `test.csv`, handling missing values, and performing transformations consistent with the dataset structure.
- **NumPy** — Numerical operations required during preprocessing and feature handling.

Preprocessing and Modeling Layer:

- **Scikit-learn** — Main library for preprocessing steps (encoding, imputation) and model training. Includes algorithms such as Logistic Regression, Decision Trees, Random Forests, and others suitable for a binary classification task.

Evaluation Layer:

- **Scikit-learn metrics** — Used to compute accuracy and other internal metrics that help quantify sensitivity and model stability.
- **Matplotlib / Seaborn** — Visualization tools used to analyze feature behavior, correlations, and sensitivity patterns within the system.

Interfaces and Reporting Layer:

- **Jupyter Notebooks or Kaggle Notebooks** — Environment for running experiments, documenting the system process, and generating reproducible analyses.

Version Control:

- **Git + GitHub** — Ensures traceability of system modifications, updates to preprocessing steps, and model iterations. Supports collaboration and workshop submission.

5.2 Justification

The selected stack aligns with the system requirements defined earlier:

- **Modularity:** Each tool contributes to a specific subsystem (data ingestion, preprocessing, modeling, evaluation), matching the architecture defined in Item 3.
- **Reproducibility:** Python, Pandas, and Scikit-learn ensure deterministic operations, reducing sensitivity to uncontrolled variability.
- **Traceability:** Git and GitHub enable continuous documentation of architectural and preprocessing changes.
- **Interpretability:** Visualization libraries support sensitivity analysis, making the internal behavior of the model understandable.

This technical stack provides the necessary foundation to implement the designed system, addressing the variability found in the dataset while ensuring clarity, structure, and maintainability across the entire workflow.